CS4433 Software Engineering: Individual Final Project

Final Report

Moderated Learning Management System (LMS)

Final submission, 2 November 2026

Student: LELIA TS'OANELO

Student no.: 202322605

Repository (code, report, figures and demo video): https://github.com/lelia-605/cs4433-lms

Abstract

This report describes the development of a Moderated Learning Management System for a secondary school, following the Waterfall model with V-Model testing over seven weeks. An administrator manages users, departments and courses; lecturers set assignments, grant extensions and mark work; Heads of Department moderate and release marks; and students submit work and see their released results. Late work is accepted but loses 10% of the maximum mark per day for up to three days, and every change to a mark is recorded in an audit log that cannot be edited. The system is a C++17 console application that keeps its data in one SQLite file and works without an internet connection. It was built in small slices on feature branches, checked by a GitHub Actions pipeline with static analysis and coverage, released as v1.0, and extended to v1.1 for change request CR1 (class-wide deadline extensions). All 19 Must requirements are met, 117 automated tests pass, and 96% of the business-logic code is covered by tests. Figures and the demo video are kept in the repository and linked from the text.

Table of Contents

Section	Page
2. Introduction	3
3. Project Management Plan	3
4. Requirements Engineering	5
5. Software Design	9
6. Implementation	12
7. Verification and Validation	14
8. Configuration Management and Quality Assurance	19
9. Deployment	23
10. Maintenance (CR1)	25
11. Professional, Legal and Ethical Issues	29
12. Conclusion	32
13. References	33
14. Appendices	35

2. Introduction

- **Background**

In many schools a lecturer's mark goes straight to the learner, late work is treated differently by each teacher, and nobody can later say who changed a mark or why. The school in this case study wants every mark checked by the Head of Department (HoD) before learners see it, one late-penalty rule for everyone, and a record of every change. Its computers are old and its internet is unreliable, so the system must run offline on one computer.

- **Aim and objectives**

The aim was to design, build, test and deliver a system that supports moderated assessment. The objectives were to:

- **O1** support four roles (Administrator, Lecturer, HoD, Student), each seeing only what it may;
- **O2** apply the agreed rules for deadlines, extensions, resubmission, late penalties, moderation and release;
- **O3** record every mark change in an audit log and protect passwords and records;
- **O4** follow the Waterfall phases with documented deliverables and supervisor gates;
- **O5** verify the system with automated tests covering at least 70% of the business logic;
- **O6** manage the code with Git and continuous integration, release v1.0 and handle a change request as v1.1.

- **Scope and approach**

In scope are the four roles, courses, assignments, extensions, submissions, marking, moderation, release and the audit log, in a console program on one computer. Out of scope, as set by the brief, are online payment, video conferencing, automated grading, plagiarism detection and web or mobile interfaces. The project followed the Waterfall model with three supervisor gates (after requirements, design and testing), and testing followed the V-Model. Sections 3 to 10 follow the phases; Section 11 discusses professional issues and Section 12 concludes. Figures are not embedded: each is a link to a file in the repository folder report/images, as the submission rules require.

3. Project Management Plan

- **Process model**

The brief sets the Waterfall model, in which each phase is finished and reviewed before the next starts. It suits this project because the requirements were fixed by the case study at the start and the supervisor gates give clear points to check the work. Its weakness is that problems found late are expensive, because earlier phases are already closed. Two things reduced that risk. First, testing followed the V-Model, so the test level for each phase was planned while that phase was written: the acceptance tests come from the stakeholder needs, the system tests from the use cases, and the integration and unit tests from the design. Second, within the implementation phase the code was built in thin slices, each tested and merged on its own, so the system worked at every step and design problems showed up early rather than at the end.

- **Feasibility**

The project was judged feasible on five dimensions. Technically, the tools are mature and free, and SQLite suits old computers because it needs no server. Operationally, a menu-driven console with one menu per role mirrors how the school already works. Economically, every tool is free and the

system runs on existing computers, so the only cost is about 235 hours of developer time. Legally, learners' data is protected by role-based access, salted password hashes and collecting only the data needed. The schedule was the biggest risk: the design phase was only about a week and a half, and the Waterfall gate meant coding could not start before the design was approved.

Dimension	Verdict	Main condition
Technical	Feasible	Complete the Week 1–2 technical spike (database, hashing, one test in CI)
Operational	Feasible	Keep the interface simple and menu-driven
Economic	Feasible	Track effort against the 235-hour estimate
Legal	Feasible	Build data-protection controls into the requirements
Schedule	Feasible, low float	MoSCoW scope control; drop Could items first

Phase	Estimated effort (hours)
Planning and management	10
Requirements engineering	25
System design	35
Implementation	55
Verification and validation	40
Configuration management and QA	10
Deployment	15
Maintenance (CR1)	15
Completion (report, ethics, viva)	20
Total	235

- **Work breakdown and schedule**

The work breakdown structure (WBS) splits the project into nine phases, each with work packages that end in a deliverable. The schedule is shown in half-weeks in the Gantt chart.

Figure 1: Work breakdown structure. See fig01-wbs.png

Figure 2: Gantt chart with critical path and milestones. See fig02-gantt-chart.png

Milestone	Target	Exit criterion
Gate 1: requirements	End of W2	SRS, stakeholder analysis and use case model reviewed
Gate 2: design	End of W3	Architecture, UML models and database design approved
Gate 3: testing	End of W6	All Must requirements tested; no open critical defect
Release v1.0	End of W6	Tagged release, installation guide and manuals

Milestone	Target	Exit criterion
Release v1.1 (CR1)	End of W7	Change implemented, regression suite passing
Submission and viva	End of W7	Report and repository link submitted

The critical path runs from feasibility through stakeholder analysis, the SRS and use cases, architecture and class design, Gate 2, the data-access layer, the domain rules, unit, integration and system tests, release v1.0, the CR1 impact analysis, v1.1 and regression tests to the viva. It has no float, so any delay in it delays the end of the project.

- **Risk management**

Risks were scored for likelihood (L) and impact (I) from 1 to 5, with exposure = L × I. The baseline register had eleven risks; the highest were the late start and compressed schedule (R1, 20) and loss of work (R3, 15). R1 did happen: the project started late, so all Must requirements were built first and Could requirements were dropped, as planned. The updated register with the final status of every risk is in Appendix D.

The plan was used as a working tool rather than written once. At each gate the completed work packages were compared with the Gantt chart, and the risk register was reviewed. Because the project started late, the later phases were compressed: deployment, maintenance and the final report were done in the last week. The MoSCoW priorities decided what to keep, so every Must requirement was delivered and only the two Could requirements were left out.

4. Requirements Engineering

- **Stakeholder analysis**

Stakeholders were taken from the five interview statements, plus the administrator and two indirect stakeholders. The Principal and the HoD have the most influence, so their rules carry the most weight where statements conflict, but other needs were met by adding visibility or flexibility rather than being ignored.

The administrator gave no statement, but is a direct user who sets up every term, so their needs were added from the brief. Parents and the data protection regulator do not use the system, but they are affected by how learners' data is handled, so they shaped the security and privacy requirements.

Stakeholder	Main needs	Influence
Principal	Simple system; no mark seen before the HoD has checked it	High
Head of Department	See every mark before release; one consistent late-penalty rule	High
Lecturer	Quick marking; extensions for good reasons; written feedback	Medium
Students (representative)	Resubmit before the deadline; feedback; late work not rejected	Medium
IT officer	Works offline on old computers; protected passwords; fast with 800 students	Medium
Administrator	Simple management of accounts, courses and enrolment	Medium

Stakeholder	Main needs	Influence
Parents (indirect)	Privacy of children's data; fair assessment	Low
Regulator (indirect)	Lawful processing under the Data Protection Act, 2011	Low

- **Conflicts and their resolution**

Twelve conflicts, ambiguities and gaps were found in the statements. The most important are shown; each became one or more requirements.

The two biggest conflicts were solved by meeting both sides rather than choosing one. For C1, the Principal's rule wins because moderation is the purpose of the system, but the lecturer gets what they really need, which is to know where each piece of work stands, through a live status list. For C2, the HoD's penalty is kept, but nothing is ever rejected: late work is stored and seen by the HoD, so the school can still deal with special cases. Ambiguities such as C3 and C4 were settled with exact rules and worked examples, because a rule that two people can read differently cannot be tested. These decisions were presented at Gate 1 as assumptions for the supervisor to confirm.

ID	Problem	Resolution	Becomes
C1	Principal: no mark before HoD check. Lecturer: release on marking.	Release only after HoD approval; lecturers see the live status of each submission.	FR14, FR18, FR21
C2	HoD: zero after 3 days. Students: late work must not be rejected.	Late work is always accepted; after 3 days it receives zero but is recorded.	FR9, FR12
C3	"10% of the available mark": of the maximum or of the mark earned?	10% of the maximum mark per day, taken from the raw mark, never below zero.	FR11, BR1
C4	What is "a day late"?	Each started 24-hour period after the effective deadline; one second late is day 1.	FR11, BR1
C5	Extensions against a fixed penalty rule.	The penalty counts from the student's extended deadline; extensions are recorded and visible to the HoD.	FR7, FR11
C6	Which version is marked? Can late work be resubmitted?	Resubmit only before the deadline; the latest version is marked; late work is final.	FR10
C7	Must the HoD see every mark, and change marks?	Every mark is moderated; adjustments need a reason and are audited; bulk approval allowed.	FR15–FR17, FR19
C9	"Respond quickly" cannot be tested.	Measurable targets: login within 2 s, lists within 1 s, with 800 students.	NFR1, NFR2
C11	Simple versus secure.	One login screen and role menus; security is built in and invisible.	FR1, NFR4–NFR6

- **Software requirements specification**

The SRS follows the structure of ISO/IEC/IEEE 29148. Requirements are numbered, prioritised with MoSCoW and each can be verified by a test (T), demonstration (D) or inspection (I). A submission moves through a fixed life cycle: Submitted or Late, then Marked, Moderated and Released, and

cannot skip a state. FR25 and the new wording of BR2 were added in release v1.1 for change request CR1.

The system is a standalone program for one school computer with no internet. Its users are the four roles, all of whom log in with a user name and password and see only their own actions. The main constraints are C++17, CMake, SQLite reached only through a data-access layer, salted password hashes and the seven-week schedule. The main assumptions are that each assignment has one deadline, that time comes from the computer's clock in UTC, that the administrator enters accurate data and that each course belongs to one department. The specification has 25 functional requirements (20 Must, 3 Should, 2 Could) and 11 measurable non-functional requirements, more than the 12 and 6 the brief asks for. Vague wishes such as "fast" were turned into numbers, for example a login within two seconds with 800 students stored, so that each one could be checked by a test.

ID	Business rule
BR1	Late penalty = 10% of the maximum mark × days late (days 1–3), from the raw mark, never below zero; beyond 3 days the mark is zero.
BR2	Effective deadline = the latest of the assignment deadline, the class extension and the student's own extension (changed in v1.1).
BR3	Resubmission only before the effective deadline; only the latest version is marked.
BR4	Students see a mark and its feedback only when the submission is Released.
BR5	Every mark is moderated by the HoD of the course's department before release.
BR6	Every creation or change of a mark is audited: who, when, old value, new value, reason.

ID	Functional requirement	MoSCoW	UC
FR1	Authenticate users by username and password; generic message on failure	Must	UC01
FR2	Show each user only the menu options and data allowed for their role	Must	UC01
FR3	Administrator creates, edits and deactivates user accounts	Must	UC02
FR4	Administrator creates and edits departments and courses	Must	UC03
FR5	Administrator assigns lecturers to courses and enrols students	Must	UC04
FR6	Lecturer creates an assignment with title, deadline and maximum mark	Must	UC05
FR7	Lecturer grants an individual extension, recording who, when and why	Must	UC06
FR8	Student views enrolled courses and assignments with effective deadlines	Must	UC12
FR9	Student submits work; time recorded; Submitted or Late; late work never rejected	Must	UC13

ID	Functional requirement	MoSCoW	UC
FR10	Student resubmits before the effective deadline; earlier versions kept	Must	UC14
FR11	Late penalty calculated by BR1 and BR2	Must	UC15
FR12	Mark of zero for work more than 3 days late, still recorded	Must	UC15
FR13	Lecturer enters a mark (not above the maximum) and feedback	Must	UC07, UC08
FR14	The submission life cycle is enforced	Must	UC07, UC09, UC11
FR15	HoD views all marked submissions of the department, with extensions and penalties	Must	UC09
FR16	HoD approves a mark or adjusts it with a reason	Must	UC09, UC10
FR17	HoD releases moderated marks	Must	UC11
FR18	Student sees mark and feedback only when Released	Must	UC16
FR19	Every creation or change of a mark recorded in an audit log	Must	UC17
FR20	HoD and administrator view the audit log for a submission	Should	UC17
FR21	Lecturer sees the state of every submission in their courses	Should	UC07
FR22	Administrator resets a user’s password	Should	UC02
FR23	Users filter lists	Could	UC12
FR24	HoD exports released marks to CSV	Could	UC11
FR25	Lecturer extends the deadline for a whole class (CR1, v1.1)	Must	UC06

ID	Non-functional requirement (measurable)	MoSCoW
NFR1	Login within 2 seconds with 800 students stored	Must
NFR2	A student’s assignment list within 1 s; HoD list of 800 within 3 s; 25,000 submissions	Must
NFR3	At least 800 students, 50 lecturers, 30 courses and 25,000 submissions without data loss	Must
NFR4	No plain-text passwords; salted Argon2id hashes (libsodium)	Must
NFR5	All database access through the data-access layer with parameterised queries	Must
NFR6	100% of role-violation test cases are denied	Must
NFR7	Invalid input or a database error gives a message and never crashes the program	Must
NFR8	Runs fully with no internet connection	Must
NFR9	Common tasks within 4 menu selections; a user manual per role	Should
NFR10	At least 70% line coverage of the business logic; no warnings with -Wall -Wextra	Must

ID	Non-functional requirement (measurable)	MoSCoW
NFR11	Exactly one audit entry per mark change; audit entries cannot be edited or deleted	Must

- **Use case model**

The use case diagram shows the four actors, each a kind of User who logs in (UC01). Marking and adjusting always include writing an audit entry, and releasing includes moderation; resubmitting and applying a late penalty extend submitting. Three use cases are fully specified in Appendix B: Submit assignment (UC13), Mark submission (UC07) and Moderate and release marks (UC09–UC11), each with its main, alternative and exception flows.

Figure 3: Use case diagram. See fig03-use-case-diagram.png

5. Software Design

One decision shapes the whole design: only the raw mark is stored. The final mark is worked out when needed from the raw mark, the days late and the maximum mark, so it can never go out of date if the penalty rule changes, and the database stays normalised.

- **Architecture**

The system has four layers: presentation, business logic, data access and external resources. Each layer calls only the one below, and the business layer reaches the data only through repository interfaces, so the rules can be tested without the console or a real database.

Figure 4: Layered architecture and components. See fig04-architecture.png

Criterion	Layered	MVC	Client–server	Single layer
Testability of business logic (NFR10)	5	3	4	1
Works offline on old computers (NFR8)	5	5	1	5
Simplicity for one developer in 7 weeks	4	3	2	5
Central security and access control (NFR5, NFR6)	5	3	4	2
Ease of change, e.g. CR1	5	4	4	1
Total (out of 25)	24	18	15	14

The layered style was chosen because the business rules are the heart of the system and must be tested. MVC was rejected because it suits graphical interfaces with several views of one model; with one console the controller tends to absorb the rules. A client–server design with a database server would break the offline requirement, and putting SQL in the menu code cannot be unit tested and breaks the brief’s data-access rule.

- **Class design**

The class diagram is drawn as four views of one model. User is an abstract class with three pure virtual operations (role, allowedActions, menuTitle), implemented by Administrator, Lecturer, HeadOfDepartment and Student, so the console shows the right menu without checking the type.

The domain view shows departments, courses, enrolments, assignments, extensions, submissions with their versions, and audit entries. The services view shows that every service depends on interfaces, and the Observer view shows how mark changes are recorded. Figure 9 is the services view as built in release v1.1, including MarkingService, the shared transaction and class extensions (Section 6 explains the changes).

Figure 5: Class diagram, View 1: User hierarchy and Factory. See fig05-class-view1-users.png

Figure 6: Class diagram, View 2: domain entities. See fig06-class-view2-domain.png

Figure 7: Class diagram, View 3: services, Strategy and Repository (design). See fig07-class-view3-services.png

Figure 8: Class diagram, View 4: Observer (design). See fig08-class-view4-observer.png

Figure 9: Class diagram, services view as built in release v1.1. See fig09-class-services-v1.1.png

The four patterns work together. The Factory turns a row from the user table into the right kind of User, so the rest of the program never checks the role text. The Repository interfaces keep every SQL statement in one layer, which is why the services can be tested on a fresh in-memory database. The Strategy holds the disputed late-penalty rule in one small class, so the HoD's rule could be replaced without touching the marking code. The Observer makes auditing automatic: a service only reports that a mark changed, and every attached observer reacts, so no service can forget to write the audit entry.

Pattern	Where used	Why	Alternative rejected
Strategy	IPenaltyStrategy, StandardPenaltyStrategy	The late rule is the most disputed rule, so it can be changed or tested alone	if/else in the service
Repository	Five repository interfaces and their SQLite classes	All SQL in one layer; services testable	SQL inside services
Observer	ISubmissionObserver, AuditObserver, StatusObserver	Every mark change is audited whichever service makes it (FR19)	Audit calls copied into each service
Factory	UserFactory	Builds the right User subclass from the stored role	Switch statements on the role spread through the code

SOLID principle	How the design follows it
Single responsibility	Each class has one reason to change: submission rules, marking, moderation, login, persistence, auditing and input checks are separate classes.
Open/closed	A new penalty rule is a new strategy class; a new reaction to mark changes is a new observer. CR1 was added without changing the penalty or life-cycle code.
Liskov substitution	Every User subclass and every repository implementation can be used where its base type is expected.
Interface segregation	Small interfaces: IPenaltyStrategy and ISubmissionObserver have two operations each; one repository per aggregate.

SOLID principle	How the design follows it
Dependency inversion	Services depend on interfaces; concrete SQLite, libsodium and clock classes are chosen in one place (LmsServices).

Other decisions: every service checks the user's allowed actions and the ownership rule (the lecturer teaches the course, the HoD's department owns it, the student is enrolled); errors are exceptions that the menu shows as messages; every write runs in one transaction together with its audit entry; and all times are UTC, read through an IClock interface so tests can fix the time.

- **Behavioural models**

The state machine allows only the moves of the life cycle; any other move is refused. The activity diagram shows the whole process across the four roles, and the two sequence diagrams show submitting (normal, late and resubmission paths) and moderating and releasing, where the observers write the audit entry and the new state inside the same transaction.

Figure 10: State machine of a submission. See fig10-state-machine.png

Figure 11: Activity diagram with swimlanes. See fig11-activity-swimlanes.png

Figure 12: Sequence diagram: submit assignment. See fig12-sequence-submit.png

Figure 13: Sequence diagram: moderate and release marks. See fig13-sequence-moderate-release.png

The submit sequence looks up the effective deadline before anything else, so an extension is always taken into account, and then the number of days late decides which branch runs. The moderation sequence was added for the final report because the design only had one sequence diagram; it is drawn from the code of release v1.1 and shows the most important safety property of the design: the new mark, its audit entry and its new state are saved in one transaction, so a failure part-way leaves no trace.

- **Database design**

The database has ten tables in release v1.1 (nine in v1.0, plus class_extension). It is in third normal form: every column is atomic, versions have their own table instead of repeated columns, and no non-key column depends on another; the final mark is derived and not stored. Two choices are deliberate: all user types share one table with a role column, because the subclasses add almost no data, and days_late is stored because it is a fact about the submission when it was received. The data dictionary is in Appendix C and the schema is the file schema.sql in the repository.

Figure 14: Entity–relationship diagram. See fig14-erd.png

The SQL schema used by the program: schema.sql

Class	Table	Note
User and its four subclasses	app_user	Single-table inheritance; role is the discriminator
Department, Course, Enrolment	department, course, enrolment	Direct mapping; composite key prevents duplicate enrolments

Class	Table	Note
Assignment, Extension, ClassExtension	assignment, extension, class_extension	One extension per student and assignment; one class extension per assignment
Submission, SubmissionVersion	submission, submission_version	State as a checked text column; final mark not stored
AuditEntry	audit_log	Append-only, enforced by triggers

6. Implementation

The system was built in C++17 after the design passed Gate 2. Work started with a walking skeleton that ran through every layer, and the rest was added one thin slice at a time, each on its own branch with its own tests. The final build has no compiler warnings with -Wall -Wextra -Wpedantic, and all 19 Must requirements and the Should requirements FR20–FR22 are implemented.

Area	What was built
Persistence	DatabaseConnection (RAII), parameterised Statement, Transaction, ITransactionRunner, schema.sql and five Sqlite*Repository classes; lms.db created on first start
Security	Argon2id hashing, generic login failure, role and ownership checks in every service, append-only audit log, password reset
Domain	Abstract User and four subclasses, UserFactory, Course, Assignment, Extension, ClassExtension, Submission (state machine), SubmissionVersion, AuditEntry
Business rules	Assignment, Submission, Marking and Moderation services; IPenaltyStrategy; Observer for the audit log and status
User interface	ConsoleApp with one login screen and four role menus; InputValidator

- **Structure**

The folders follow the layers of the architecture. The core code is built as a library, lms_core, so the program and the tests use exactly the same code.

```
include/lms/  domain/  services/  security/  data/  app/  ui/
src/         .cpp files and main.cpp
tests/       11 test files (117 tests)
schema.sql   database schema;  docs/  coding standard, design changes, quality reports
report/      final report, images and demo video link
```

- **Technology decisions**

Choice	Why	Alternative rejected
C++17, CMake	Required by the brief; one CMake file builds the library, program and tests, and the CI pipeline uses the same file	Hand-written Makefile: not portable

Choice	Why	Alternative rejected
SQLite	Single-file database with no server; works offline on old computers (NFR8)	MySQL or another server database: needs a running server
libsodium (Argon2id)	Widely reviewed library; Argon2id is memory-hard, so it resists GPU cracking, and the salt is stored inside the hash string, so salting cannot be done wrongly; hashing takes about a tenth of a second, well inside NFR1	bcrypt (older, not memory-hard); plain SHA-256 (far too fast); own cryptography (unsafe)
GoogleTest	Well-known C++ framework; CMake downloads it; works with ctest and CI	Catch2: also suitable

- **Persistence and security**
- **Parameterised queries only:** values are bound to a prepared statement, so input is never part of the SQL text; a test confirms that an injection attempt is treated as plain text.
- **One transaction per change:** marking and moderation save the mark and notify the observers inside one transaction; AuditObserver writes the audit entry and StatusObserver saves the new state. If any step fails, nothing is saved (NFR11).
- **Rules in the database too:** CHECK constraints, one HoD per department and triggers that block editing or deleting the audit log are a second line of defence behind the services.
- **Access control:** each service checks the role and ownership before acting, and students see marks only after release.
- **Coding standard:** docs/CODING_STANDARD.md sets naming, layout (clang-format), RAII with no raw new or delete, exceptions for errors, SQL only in repositories, and limits of 40 lines and a complexity of 10 per function.
- **Key rules in code**

Three small pieces of code carry most of the business rules. The days late are the number of started 24-hour periods after the effective deadline, worked out in whole seconds, so a submission exactly on the deadline is on time and one second later is one day late. The penalty strategy takes 10% of the maximum mark for each day up to three, rounds the deduction to the nearest whole mark and never lets the final mark go below zero; after three days the mark is zero. The effective deadline is the latest of the assignment deadline, the class extension and the student’s own extension, so an extension can only ever give a student more time. Each of these has boundary tests, described in Section 7.

Each slice of work followed the same loop: write the code and its tests, build with no warnings, run the whole test suite, review the change against the design, and only then open a pull request. When the implementation was compared with the SRS and the class diagram near the end, several gaps were found, including the missing Observer pattern and some use-case flows. They were fixed in a dedicated slice before release v1.0, and the remaining differences are listed below.

- **Differences from the design**

Some details changed during coding. Each is recorded here and in docs/DESIGN_CHANGES.md, and the class diagram as built is Figure 9.

Design	As built	Reason
SubmissionService also marks	Separate MarkingService	Single responsibility: submission and marking rules change for different reasons
Effective deadline in ISubmissionRepository	AssignmentService::effectiveDeadline	Extensions belong to assignments; CR1 still changed only one place
Observers share the transaction	New interface ITransactionRunner	Services run the save and the observers in one transaction without SQL
RoleMenus class	Four menu methods of ConsoleApp	Simpler for a console interface
(not in design)	approveAll, partial release, LmsServices, prepareDatabase	UC09 A2, A3, E3; wiring in one place; first start

- **The running program**

The screenshots in the repository show every step of a full term, from the first start to a released mark. The most important ones are linked here; the full list is in report/README.md.

Figure 15: Lecturer marks late work and the penalty is applied. See screenshot-08-lecturer-mark-late-work.png

Figure 16: Release refused while work is not moderated. See screenshot-11-hod-release-refused.png

Figure 17: HoD adjusts a mark with a reason; the audit trail. See screenshot-12-hod-adjust-release-audit.png

Figure 18: Student sees the final mark after release. See screenshot-13-student-result-after-release.png

All screenshots: report/README.md

7. Verification and Validation

Verification checks that the system is built right, against the design; validation checks that the right system is built, against what the stakeholders need. Both were planned from the start. The test plan was written in Week 3, next to the design, and every requirement in the SRS was given a way to be checked. Tests were written together with the code in each slice rather than afterwards, and the whole suite ran on every push, so a change that broke an earlier feature was caught within minutes. Time is the hardest thing to test in this system, because deadlines and late days depend on the clock. The services therefore read the time through an IClock interface, and the tests use a fixed clock, which makes it possible to test a submission exactly on the deadline, one second after it, and exactly 72 hours after it.

Testing followed the V-Model: each development artefact has a test level that checks it. Verification (“are we building the product right?”) was done with unit and integration tests against the design. Validation (“are we building the right product?”) was done with system tests derived from the use cases and acceptance tests derived from the stakeholder needs.

Item	Result
Automated tests	103 GoogleTest tests: 13 unit, 78 integration, 12 system
Result on release v1.0	103 passed, 0 failed (about 27 s), locally and in GitHub Actions
Run with networking disabled (NFR8)	103 passed, 0 failed
Line coverage, business-logic layer (NFR10)	95% (target 70%); branch coverage 88%
System test cases from use cases	27 cases covering every main, alternative and exception flow of UC01, UC07, UC09–UC11 and UC13; 27 passed (Appendix E)
Acceptance test cases	9 cases, one per stakeholder concern; 9 passed
Defects logged	14, all fixed and retested (D13 and D14 in v1.1)

- **Test Plan**
- **V-Model levels**

Level	Verifies	How	Tests
Unit	Detailed design: single classes and functions (class diagram)	One function or class with no database, e.g. the penalty rule, days late, state machine, validator	13
Integration	Architecture: services working with the SQLite repositories	A service with real repositories on a fresh in-memory database for every test	78
System	SRS: functional and non-functional requirements, use cases	Whole sessions typed into the console; a database file reopened; full-size data for timing	12
Acceptance	Stakeholder needs and the resolved conflicts (Phase 2)	Each stakeholder concern checked on the final build (Section 5)	9 cases

- **Entry and exit criteria**

Level	Entry criteria	Exit criteria
Unit	Class written and compiles with no warnings	All unit tests pass; every boundary in the partition table has a test
Integration	Unit tests pass; schema.sql loads	All tests pass; business-logic line coverage is at least 70%; each FR has at least one test
System	Integration tests pass; console menus built	Every main, alternative and exception flow of the three fully dressed use cases has a passing case; NFR1–NFR3 timings met
Acceptance	System tests pass; release candidate built	Every stakeholder concern passes; no open defect of high or medium severity

- **Test environment**

- **Machines:** Ubuntu on WSL (development laptop) and GitHub Actions ubuntu-latest (CI).
- **Tools:** GoogleTest 1.14 with ctest; gcov and gcovr for coverage.
- **Isolation:** every test builds its own in-memory SQLite database from schema.sql, so tests cannot affect each other.
- **Time:** services read the time through IClock. Tests use FixedClock, so deadline boundaries such as “exactly 72 hours late” are tested to the second.
- **Equivalence partitioning and boundary values**

Each input was split into classes that the system should treat the same way (equivalence partitions). Tests use the values on and next to each boundary, where mistakes are most likely.

Input	Partitions	Values tested	Test
Submission time vs effective deadline	On time | 1–3 days late | more than 3 days late	deadline −1 s, deadline, +1 s, +24 h, +24 h +1 s, +72 h, +72 h +1 s, +240 h	DaysLate.BoundaryValues; LateDaysBoundaryValues
Final mark (max 50, raw 40)	0, 1, 2, 3 days late | more than 3 days	0→40, 1→35, 2→30, 3→25, 4→0; raw 10 at 3 days → 0 (never below zero)	Penalty.SrsExampleTable; LatePenaltyTable
Penalty rounding	Below .5 | .5 and above	max 33 → 3; max 35 → 4	NeverBelowZeroAndRounds
Mark entered (max 50)	Below 0 | 0 to max | above max	−1, 0, 50, 51	MarkBoundaryValues; AdjustBoundaryValues
Maximum mark	Below 1 | 1 to 1000 | above 1000	0, 1, 1000, 1001	RejectsInvalidAssignmentInput
Submission text	Blank | 1 to 10,000 characters | longer	"", spaces, 10,000, 10,001	RejectsEmptyOrTooLongContent
Feedback	Blank | 1 to 2,000 characters | longer	spaces, 2,000, 2,001	FeedbackIsRequiredAndLimited
Extension deadline	Not later than current deadline | later	equal, earlier, later	ExtensionRulesAreEnforced
Date typed by the user	Valid date and time | wrong format | impossible date	2026-11-01 10:00; 1 Nov 2026; 2026-02-30 10:00; 25:00	Validator.ParsesNumbersAndDates
Password	Shorter than 8 | 8 or more	5 and 9 characters	RejectsInvalidInput; ResetPasswordChangesWhichPasswordWorks
Life-cycle transition	Allowed | not allowed	all 25 pairs of the 5 states	StateMachine.OnlyAllowedTransitions

- **Automated Test Results**

All tests are run by ctest locally and by the CI pipeline on every push. The table lists the results for release v1.0.

Test file	Level	Requirements	Tests	Result
test_penalty.cpp	Unit	FR11, FR12, BR1	3	Pass
test_users.cpp	Unit, integration	FR1, FR2, NFR4–NFR6, NFR11	8	Pass
test_admin.cpp	Integration	FR3–FR5, FR22	11	Pass
test_assignments.cpp	Unit, integration	FR6–FR8, BR2	12	Pass
test_submissions.cpp	Integration	FR9, FR10, FR12, FR21, BR2, BR3	18	Pass
test_marking.cpp	Integration	FR11, FR13, FR19, FR20, BR1, BR6, NFR11	14	Pass
test_moderation.cpp	Integration	FR14–FR18, BR4, BR5	13	Pass
test_design.cpp	Unit, integration	FR4, FR14, FR15, FR19, NFR7, NFR11	12	Pass
test_console.cpp	System	FR1, FR2, FR18, NFR3, NFR7, NFR9	8	Pass
test_performance.cpp	System	NFR1–NFR3	4	Pass
Total			103	103 pass

Coverage: the business-logic layer has 95% line coverage (496 of 520 lines) and 88% branch coverage. The whole program has 84%. The lowest file is the console (55%), because the system tests use only the main menu paths. The full per-file report is docs/quality/test-coverage.txt and is also kept by every CI run.

Performance (NFR1–NFR3): with 800 students, 50 lecturers, 30 courses and 25,600 submissions stored, login took about 0.25 s (limit 2 s), a student's assignment list about 0.17 s (limit 1 s) and the HoD's list of 800 marks about 0.2 s (limit 3 s).

Offline (NFR8): the whole suite was run inside a network namespace with no network; all 103 tests passed.

- **Acceptance Tests**

Each acceptance case checks one stakeholder concern or one conflict resolved in Phase 2. They were run on release v1.0, partly through the console and partly through the automated tests.

ID	Stakeholder concern	Evidence	Result
AT-1	Principal: students see no mark until the HoD has checked it (C1)	StudentSeesNothingBeforeRelease; console shows "not released yet"	Pass
AT-2	HoD: 10% of the maximum per day for 3 days, then zero (C2, C3)	LatePenaltyTable	Pass
AT-3	Students: late work is never rejected (C2)	VeryLateWorkIsStillAccepted	Pass
AT-4	Lecturer: an extension removes the penalty for that student only (C5)	ExtensionMeansNoPenalty; ExtensionRemovesLatenessForThatStudentOnly	Pass

ID	Stakeholder concern	Evidence	Result
AT-5	HoD: every mark is moderated before release, no sampling (C7)	ReleaseNamesThePendingSubmissions	Pass
AT-6	Student: feedback is hidden until release, like the mark (C8)	StudentSeesNothingBeforeRelease	Pass
AT-7	IT officer: fast enough with a full school (C9)	PerformanceFixture (4 tests)	Pass
AT-8	IT officer: works on one computer with no internet (C10)	Full suite with networking disabled	Pass
AT-9	Principal and IT officer: simple to use but secure (C11); submit, mark and release each take one menu choice after login (NFR9 allows 4)	RolesOnlySeeTheirOwnMenu; PasswordIsNotStoredInPlainText; console walkthrough	Pass

- **Defect Log**

Every problem found by a test, a review or a tool was logged. Severity: High = data loss or a Must requirement not met; Medium = a requirement or design element only partly met; Low = quality or style.

ID	Found by	Severity	Defect and cause	Fix	Retest
D01	Compiler	Low	Warning that toIso could cut off the date text: buffer of 32 bytes	Buffer 64 bytes	Build: 0 warnings
D02	Code review	High	All data lost when the program closed: main.cpp used an in-memory database	Database file lms.db; schema created on first start	DataSurvivesRestartAndAdminIsNotRecreated: Pass
D03	Code review	High	Admin features worked only in the services: no console menus, so FR2 could not be demonstrated	ConsoleApp with four role menus	ConsoleFixture (8): Pass
D04	Review vs SRS	Medium	Release always released part of the marks; UC09 E3 needs a refusal while work is pending	release() refuses and lists the pending ones; partial release only on request	ReleaseNamesThePendingSubmissions: Pass
D05	Review vs SRS	Medium	HoD list did not show the lecturer and the extension (FR15)	Fields added to ModerationItem and the console	ModerationListShowsLecturerAndExtension: Pass
D06	Review vs SRS	Medium	No way to edit a department (FR4)	renameDepartment added	AdminRenamesDepartment: Pass
D07	Review vs design	Medium	Observer pattern from the class diagram not implemented; audit written directly by the repository	ISubmissionObserver, AuditObserver, StatusObserver in one transaction	AuditObserverWritesExactlyOneEntryPerChange: Pass

ID	Found by	Severity	Defect and cause	Fix	Retest
D08	Review vs SRS	Low	Administrator could not view the audit log (FR20, Should)	ViewAuditLog permission and menu option	AuditTrailVisibility: Pass
D09	Review vs SRS	Medium	No bulk approval (UC09 A2)	approveAll()	ApproveAllApprovesOnlyMarkedWork: Pass
D10	Test review	Low	A boundary test re-ran the set-up inside its loop, so later cases did not test what they claimed	Data reset between cases instead	LateDaysBoundaryValues: Pass
D11	cppcheck	Low	Six style findings (const parameters, raw copy loop)	See Phase 6, Section 3	cppcheck: 0 findings
D12	lizard	Low	Three functions over the size or complexity limits	Split into smaller functions	lizard: 0 over limits; all tests pass
D13	Use-case walkthrough	Low	UC13 A3: the console says how many days late but does not warn that the mark will be zero	Fixed in v1.1: zero-mark warning added	ConsoleWarnsAboutZeroMarkAndShowsWorkBeforeMarking: Pass
D14	Use-case walkthrough	Low	UC07 step 3: the marking option does not display the submitted text before the mark is entered	Fixed in v1.1: work shown before the mark	ConsoleWarnsAboutZeroMarkAndShowsWorkBeforeMarking: Pass

High and medium defects were all found before release v1.0 and were fixed and retested. D13 and D14 only affected what the console showed and were fixed in release v1.1. Most defects were found by reviewing the code against the SRS and the design, not by failing tests, which shows that reviews and tests find different kinds of problems.

8. Configuration Management and Quality Assurance

All code is in one Git repository on GitHub. Every change was made on a feature branch, checked by the CI pipeline and merged by pull request. Quality was checked with a static analyser, code metrics, test coverage and a self code review against a checklist. The figures in this section are for release v1.0; Section 10 gives the same checks for v1.1.

- **Branching strategy**

A simple feature-branch strategy was used, because the project has one developer and needs a main branch that always works for the demo. Git Flow, with separate develop and release branches, was rejected as too heavy for one person and seven weeks.

- **main:** always builds and passes all tests. After the initial skeleton commit, nothing was committed to main directly.
- **feature/<name>:** one branch per slice of work, for example feature/submissions. It is pushed to GitHub, a pull request is opened and the CI checks must pass before it is merged.

- **Tags:** v1.0 marks the release for deployment (Phase 7); v1.1 marks the CR1 change (Phase 8).
- **Commit messages:** imperative and about one change, for example “Add marking, feedback, late penalty and audit log”.
- **Not under version control:** build folders, database files (*.db, *.sqlite) and coverage output, listed in .gitignore, so no test data or personal data is pushed.
- **Change history**

Each row is one pull request. The test count shows that the suite grew with every change and never went down.

PR	Branch	Change	Tests
-	main	Initial commit: walking skeleton (database layer, hashing, User classes, login, penalty rule)	11
#1	feature/admin-services	Admin services, course repository and tests	22
#2	feature/assignments	Assignments, extensions, student views, IClock	34
#3	feature/submissions	Submissions, versions and late rules	52
#4	feature/marking	Marking, feedback, late penalty and audit log	66
#5	feature/moderation	HoD moderation, release and student results	79
#6	feature/console-and-design	Console application, database file, Observer pattern and other design fixes, performance tests	103
#7	feature/quality	cppcheck and coverage in CI, review fixes, quality reports	103
#8	feature/cr1-class-extension	CR1 class-wide extensions; D13 and D14 fixed (v1.1)	117

Problem met: the internet connection dropped while slice 6 was being pushed, so its commit stayed on the laptop and pull request #6 was not created at first. The error was found because git log still showed pull request #5 at the top. Slices 6 and 7 were then merged together in pull request #6. Since then, every merge is checked with git log before the next slice starts.

- **Continuous integration**

GitHub Actions runs the pipeline in .github/workflows/ci.yml on every push and every pull request. It has two jobs that run at the same time.

Job	Steps	Fails when
build-and-test	Install SQLite and libsodium; configure with CMake; build; run all tests with ctest	the code does not compile or any test fails
quality	Run cppcheck; build with coverage (LMS_COVERAGE=ON); run the tests; measure coverage of the business-logic layer with gcovr; keep the coverage report as a download	cppcheck reports any finding, or business-logic line coverage is below 70%

A pull request is merged only when both jobs show a green tick, so main always builds and passes the tests. This also keeps the quality rules from being forgotten as the code grows.

- **Releases**

After pull request #7 was merged, the commit on main was tagged as release v1.0:

```
git tag -a v1.0 -m "Release 1.0: all Must requirements, 103 tests"
git push origin v1.0
```

- **Static Analysis**

cppcheck was chosen because it is free, runs on Linux and in GitHub Actions without set-up, and reports real faults with few false alarms. clang-tidy was also considered; it gives more checks but needs a compile database and many settings to be useful. cppcheck was run on all files in src/ with the warning, style, performance and portability checks turned on.

File and line	Finding	Fix
ModerationService.cpp:139	useStlAlgorithm: a raw loop copies assignments into a vector	Replaced with vector::insert
SqliteAssignmentRepository.cpp:6	constParameterReference: readAssignment does not change its statement	Parameter made const Statement&
SqliteCourseRepository.cpp:84	constParameterReference: readCourse	Parameter made const
SqliteSubmissionRepository.cpp:10	constParameterReference: readSubmission	Parameter made const
SqliteUserRepository.cpp:11	constParameterReference: readUser	Parameter made const
SqliteUserRepository.cpp:23	constParameterReference: runUpdate only reads the connection	Parameter made const DatabaseConnection&

The first run found six findings, all of style severity; there were no errors and no warnings. All six were fixed and the second run found none. The CI quality job now runs cppcheck with --error-exitcode=1, so a new finding stops the merge. The full output is in docs/quality/static-analysis.txt.

- **Code Metrics**
- **Size**

Folder	Files	Lines of code	Contents
include/	39	961	Headers: domain classes, interfaces, services
src/	21	1,563	Implementation and main.cpp
tests/	10	1,362	103 GoogleTest tests
Total	70	3,886	Lines of code exclude blank lines and comment lines

The test code is almost as large as the code it tests (1,362 lines against 2,524), which shows that testing was done alongside the coding and not left to the end.

- **Complexity**

Cyclomatic complexity (CCN) counts the independent paths through a function; the coding standard limits it to 10 and function length to 40 lines. Across 232 functions the average length is 7 lines and the average CCN is 2.3. The table shows the key business functions.

Function	Lines	CCN	Requirement
SubmissionService::submit	22	8	FR9, FR10, BR2, BR3
ModerationService::release	23	8	FR17, BR5, UC09 E3
Submission::canMove	12	8	FR14
AssignmentService::grantExtension	21	7	FR7
AdminService::createUser	14	7	FR3
ModerationService::resultFor	14	6	FR18, BR4
MarkingService::changeMark	15	4	BR6, FR19
MarkingService::markSubmission	15	2	FR13, FR19
StandardPenaltyStrategy::penaltyFor	5	2	FR11, FR12, BR1
AuthService::login	5	3	FR1

The highest values (8) are in the functions that apply the most rules. They stay readable because each rule is one guard clause that throws an exception. Three functions were over the limits before the review; Section 5 shows how they were fixed.

- **Test coverage**

Part of the code	Lines covered	Branches covered
Business-logic layer (services, Submission, penalty, validator, time)	496 of 520 (95%)	572 of 646 (88%)
Data-access layer (repositories, DatabaseConnection)	440 of 483 (91%)	-
Whole program (all of src/ except main.cpp)	1,114 of 1,325 (84%)	-

NFR10 asks for at least 70% line coverage of the business-logic layer; the measured figure is 95%. The console code has the lowest coverage (55%), because the system tests drive only the main paths through the menus. The uncovered business lines are mostly repository failures that are hard to cause in a test. The per-file report is in docs/quality/test-coverage.txt.

- **Self Code Review**

The code was reviewed against the checklist below, which comes from the coding standard and the security requirements. Searches (grep), the compiler, cppcheck and lizard were used where possible, so the result does not depend on memory alone.

No.	Checklist item	Result	Action
1	SQL only in Sqlite*Repository classes (NFR5)	Pass	None
2	Every user value bound as a parameter, never joined into SQL (NFR5)	Pass: joins only combine fixed SQL text	None
3	No raw new or delete; RAII for resources	Pass	None

No.	Checklist item	Result	Action
4	Interfaces have virtual destructors; overrides marked override	Pass (11 interfaces)	None
5	Every service checks the role and ownership before acting (NFR6)	Pass	None
6	Every mark change audited in the same transaction (NFR11)	Pass	None
7	Zero compiler warnings	Failed once	Buffer in toIso enlarged (change 1)
8	Functions at most 40 lines and CCN at most 10	Failed for 3	Changes 2 and 3
9	Parameters const where possible	Failed for 5	Change 4
10	No duplicated code	Failed	Change 5
11	Requirement numbers in comments match the SRS	Failed	Change 6
12	No database or build files under version control	Failed	Change 7

- **Changes made after the review**
- **Change 1.** toIso used a 32-byte buffer and g++ warned that the text could be cut off. The buffer was enlarged to 64 bytes.
- **Change 2.** fromIso had a CCN of 14 because one condition checked six ranges. A small inRange helper reduced it to 9 with the same behaviour; the date tests still pass.
- **Change 3.** adminMenu (56 lines) and lecturerMenu (44 lines) were too long. The longer options were moved into small named functions such as lecturerMark; the menus look the same on screen.
- **Change 4.** The cppcheck findings in Section 3 were fixed.
- **Change 5.** The same isBlank function was copied in four services. It was moved into InputValidator, which the design already planned.
- **Change 6.** Several comments used old requirement numbers (for example the audit log was marked FR15 instead of FR19). They were corrected so the traceability matrix can rely on them.
- **Change 7.** The *.sqlite and *.sqlite3 patterns had dropped out of .gitignore; they were added back together with the coverage output.

After the changes all 103 tests were run again and passed, cppcheck found nothing and lizard reported no function over the limits. The review shows that automatic tools find different problems from reading the code: cppcheck found the const issues, while the duplicated code and the wrong requirement numbers were only found by reading.

9. Deployment

Deployment means getting a tested release onto the school computer in a way a technician can repeat. Each release is a tag in the Git repository, so the exact code that was tested can always be

found again: v1.0 is the first release and v1.1 adds CR1. The technician downloads the tagged release, builds it with the same CMake file that the CI pipeline uses, and runs the test suite on the school computer before first use, so any problem with the libraries on that machine shows up before staff depend on it.

The system runs on one school computer. The program, its database file and the two libraries it uses are all on that computer, so it works with no internet connection (NFR8). GitHub is used only by the developer and, once, by the technician to download the release.

Figure 19: UML deployment diagram, release v1.1. See deployment-diagram.png

Element	Type	Role
School computer	«device»	The one computer the system runs on. Users log in at it one at a time.
Ubuntu (WSL 2)	«execution environment»	Linux running inside Windows; the program is built and run here.
lms	«artifact»	The program built from the v1.1 source code.
lms.db	«artifact»	The SQLite database file with all users, courses, submissions, marks and the audit log.
schema.sql	«artifact»	Creates the tables the first time the program starts.
libsqlite3, libsodium	«library»	Database engine and password hashing (Argon2id).
USB drive	«device»	Holds the daily backup copy of lms.db.
GitHub repository and Actions	«cloud»	Holds the source code and tagged releases; builds and tests every change. Not needed while the system is in use.

This design was chosen because the school has old computers and unreliable internet (Phase 2, C10). A client–server set-up with a database server was rejected: it needs a network and a server to look after. The cost of the chosen design is that only one person can use the system at a time, which matches the scope agreed for version 1.

The installation guide for the school technician (Appendix F) was tested on a fresh copy of release v1.1: it builds, all 117 tests pass, and the first start creates the database and the administrator. A user manual for each of the four roles is in Appendix G. Release v1.0 was tagged in Git after the last v1.0 pull request was merged.

- **Release Notes**
- **Version 1.1 (tag v1.1)**
- Lecturers can extend the deadline for a whole class (Lecturer option 10), as asked in change request CR1.
- Students see on their assignment list whether a deadline was extended for the whole class or for them, together with the original date.
- When an extension is given, work handed in late but not yet marked has its days late recalculated.
- Fixed: warning that work more than 3 days late receives zero; the lecturer sees the work before marking.

- A v1.0 database is upgraded automatically on the first start. 117 automated tests pass.
- **Version 1.0 (tag v1.0)**
- First release: login with four role menus; users, departments, courses and enrolment; assignments and individual extensions; submission and resubmission with versions; late penalty; marking and feedback; HoD moderation and release; append-only audit log.
- Data kept in one SQLite file; runs with no internet connection. 103 automated tests pass.
- **Known limitations**
- One user at a time on one computer; there is no network version.
- Console only; answers are one line of text and files cannot be attached.
- Dates are typed and shown in UTC.
- Filtering lists (FR23) and exporting marks to CSV (FR24) are not included; they were Could requirements.

Upgrading follows the same steps. Before an upgrade the technician copies the database file to the backup drive; the new release then adds any new table on its first start and keeps all existing data, which was tested by opening a database made by v1.0 with v1.1. Because all data is in one file, a backup is a single copy and a restore is copying it back, which suits a school without IT staff on site every day.

10. Maintenance (CR1)

- **The Change Request**

After release v1.0 the school submitted this change request:

CR1: “A lecturer should be able to extend the deadline for a whole class, not just an individual student, and extensions should be shown on the student’s assignment list.”

It asks for two things: (1) a lecturer can give one new deadline to every student on an assignment, and (2) students can see on their assignment list that their deadline was extended. In v1.0 a lecturer had to grant the same extension to each student one at a time, and the student list only showed the word “extended”.

- **Classification**

CR1 is perfective maintenance: it adds a new feature that users asked for after release, and nothing in v1.0 was wrong. The table shows why the other types do not fit.

Type	Meaning	Fits CR1?
Corrective	Fixing a fault, where the system does not meet its requirements	No: v1.0 met FR7, which only asked for individual extensions
Adaptive	Changing the system for a new environment (operating system, hardware, law)	No: the environment is the same
Perfective	Adding or improving functions because users ask for them	Yes: a new user need that came up after using the system
Preventive	Changing the code to make future maintenance easier	No: although the change was made easy by the v1.0 design (Section 3)

Release v1.1 also closes the two open defects from Phase 5 (D13 and D14). Those two fixes are corrective maintenance and are listed separately in Section 4.

- **Impact Analysis**

Every requirement, diagram, class, table and test case was checked for the effect of CR1.

- **Requirements**

Item	Impact	Change
FR25 (new, Must)	New	A lecturer shall be able to extend the deadline for every student on an assignment in one action, recording the lecturer, date and time and reason.
BR2	Changed	Effective deadline = the latest of the assignment deadline, the class extension and the student's own extension.
FR7	Checked	Individual extensions stay; a new one must now be later than the class deadline.
FR8	Changed	The student list shows whether the deadline was extended for the class or for the student alone, and the original date.
FR9–FR12	Behaviour only	Days late and the penalty use the new effective deadline. Unmarked late work is recalculated when an extension is given.
FR15	Changed	The HoD's moderation list also shows class extensions.
NFR2	Checked	One extra lookup per assignment; the performance tests still pass well inside the limits.

- **Design documents and diagrams**

Artefact	Change
Use case diagram and UC06	UC06 Grant extension gets an alternative flow "extend for the whole class"; UC12 shows the kind of extension.
Class diagram, View 2 (domain)	New ClassExtension class (0..1 per Assignment) and ExtensionKind enumeration; AssignmentView gets extensionKind.
Class diagram, View 3 (services)	New operations: AssignmentService.grantClassExtension, classDeadline, extensionKind; SubmissionService.refreshLateness, latestVersion.
ERD, data dictionary, SQL schema	New table class_extension, linked one-to-zero-or-one with assignment.
Sequence diagram (submit)	The effective-deadline step now also reads the class extension.
State machine	No new state. A Late submission can become Submitted when an extension is recalculated; this move was already allowed.
Architecture and component diagram	No change.

- **Classes and tables**

Class or table	Change
class_extension (table)	New: assignment_id (primary key), new_deadline, granted_by, granted_at, reason
ClassExtension, ExtensionKind	New
IAssignmentRepository, SqliteAssignmentRepository	saveClassExtension, findClassExtension
ISubmissionRepository, SqliteSubmissionRepository	updateLateness
AssignmentService	grantClassExtension, classDeadline, extensionKind; effectiveDeadline changed
SubmissionService	refreshLateness; latestVersion (D14)
ModerationService	listForModeration returns the class extension
ConsoleApp	Lecturer option 10; extension shown in the student and lecturer lists; zero-mark warning (D13); submitted work shown before marking (D14)
prepareDatabase	Adds the new table to a v1.0 database on start
assignment, extension, submission (tables)	No change to their structure

- **Test cases affected**

These existing tests depend on the effective deadline, so CR1 could break them. All were re-run without any change and pass: ExtensionChangesOnlyThatStudentsDeadline, ExtensionRulesAreEnforced, SecondExtensionReplacesFirstButMustBeLater, StudentSeesOwnCoursesAndEffectiveDeadlines, ExtensionIsUsedForLateness, LatenessCountsFromTheExtendedDeadline, ExtensionRemovesLatenessForThatStudentOnly, ExtensionMeansNoPenalty, ModerationListShowsLecturerAndExtension and the eight console system tests.

- **Decisions and alternatives**

Decision	Chosen	Rejected alternative
Where to store a class extension	A new table, one row per assignment. The original deadline is kept, so it can be shown, and the record says who extended it and why.	Overwriting assignment.deadline: loses the original date and the reason. Adding an individual extension for every student: students enrolled later would miss it, and it creates many rows.
Class and individual extension together	The later of the two applies, so no student can lose time they were already given.	Class extension replaces individual ones: a student with an illness extension could end up with an earlier deadline, which is unfair.
Work already handed in late	Recalculate days late for work that is not marked yet.	Leave it late: students would keep a penalty for a deadline that no longer applies. Marked work is not changed, because that would alter a mark without the lecturer or the audit log.

Decision	Chosen	Rejected alternative
Menu position	New option added at the end of the lecturer menu (10).	Next to “Grant an extension”: would renumber options users already know.

Size of the change: about 220 lines of production code in 17 files and 224 lines of new tests. The change stayed small because the Phase 3 design kept the effective-deadline rule in one function; only that function needed a new rule, as the design predicted.

- **Implementation of Release v1.1**

CR1 was built on the branch feature/cr1-class-extension and merged by pull request #8 after both CI jobs passed. The merge commit is tagged v1.1.

- **Class-wide extension:** Lecturer menu, option 10 “Extend the deadline for the whole class”. A reason is required and the new date must be later than the current class deadline and in the future. Only the course’s lecturer can do it.
- **Shown to students:** the assignment list shows the deadline that applies, whether it was extended for the whole class or for the student alone, and the original date.
- **Shown to staff:** the lecturer’s assignment list and the HoD’s moderation list also show class extensions.
- **Fair recalculation:** when any extension is given, unmarked work that was late is checked again and its days late are updated.
- **Upgrade:** a database made by v1.0 gets the new table the first time v1.1 starts; all existing data is kept.
- **Defect D13 (corrective):** the console now warns that work more than 3 days late receives a mark of zero (UC13 A3).
- **Defect D14 (corrective):** the lecturer now sees the submitted work before entering a mark (UC07 step 3).

Output of the real program after a class extension (lecturer view, then student view):

```
 1  Essay 1  due 2030-01-04 10:00 (class extension; original 2030-01-01 10:00)  out of 50
 1  Essay 1  due 2030-01-04 10:00 (extended for the whole class; original 2030-01-01 10:00)
```

Documents updated: SRS (FR25, BR2, FR8, FR15), use case UC06, class diagram Views 2 and 3, ERD, data dictionary, SQL schema and the submit sequence diagram. The repository also contains docs/DESIGN_CHANGES.md and docs/RELEASE_NOTES.md for v1.1.

- **Regression Testing**

The full v1.0 test suite was run again on v1.1 without changing any test, to show that the change did not break existing features. New tests were added for CR1 and the two defect fixes.

Run	Tests	Result
v1.0 suite on v1.1 (regression)	103	103 pass, 0 fail, no test changed
New CR1 and defect tests	14	14 pass
Whole suite, with networking disabled	117	117 pass
cppcheck	-	0 findings

Run	Tests	Result
lizard (size and complexity limits)	-	0 functions over the limits
Coverage, business-logic layer	-	96% of lines (555 of 575), 88% of branches

- **New test cases**

Test	Checks
ClassExtensionMovesEveryStudentsDeadline	Every student gets the new date; original kept; who and why recorded
StudentEnrolledLaterAlsoGetsTheClassDeadline	A student added after the extension is included
LaterOfClassAndIndividualExtensionApplies	The later deadline wins in both orders
IndividualExtensionMustBeatTheClassDeadline	An own extension must be later than the class one
ClassExtensionRules	Other lecturer, HoD and student denied; reason required; boundaries: equal date refused, +1 s accepted
SecondClassExtensionReplacesFirstAndMustBeLater	A second extension replaces the first
ClassExtensionAfterTheDeadlineMustBeInTheFuture	Cannot extend into the past
LatenessCountsFromTheClassDeadline	Boundaries: on the class deadline is on time, +1 s is 1 day late
ExtensionRecalculatesUnmarkedLateWork	Late unmarked work becomes on time; marked work unchanged
StudentListShowsKindOfExtensionAndOriginalDeadline	Class, individual or none, with the original date
HodSeesTheClassExtensionWhenModerating	FR15 shows class extensions
LecturerExtendsWholeClassAndStudentsSeeIt	System test through the console
ConsoleWarnsAboutZeroMarkAndShowsWorkBeforeMarking	Defects D13 and D14 fixed
Version10DatabaseIsUpgradedWithoutLosingData	A v1.0 database is upgraded once and keeps its data

All 117 tests pass in the CI pipeline for pull request #8. The regression run shows that the v1.0 features still work, and the defect log from Phase 5 now has all 14 defects closed.

11. Professional, Legal and Ethical Issues

The LMS stores personal information about secondary-school learners, most of whom are children, and it decides marks that affect their results. This section discusses four issues the brief names: data protection, security of assessment records, accessibility and the fairness of the late-penalty rule. For each, it states the relevant law or principle, how the design addresses it, and what the school still has to do.

- **Data Protection of Learners' Personal Information**

In Lesotho, personal information is protected by the Data Protection Act, 2011 (Act No. 5 of 2012), which applies to public and private bodies that process information about identifiable living people. The school is the data controller and the LMS is the tool it uses. The Act requires processing for a specified purpose, limited to what is adequate, relevant and not excessive, kept accurate, kept no longer than needed and protected by security safeguards. It also gives people the right to see and correct their data, and it treats information about children as sensitive personal information, which needs extra care.

Principle	How the LMS meets it	What the school must still do
Specified purpose	Data is used only for assignments, marking and moderation. The system has no reports, marketing or sharing functions.	State the purpose in its privacy notice to learners and parents.
Minimality	Only a user name, full name, role, department, enrolments, submissions, marks, feedback and the audit log are stored. No ID numbers, addresses, dates of birth or health data.	Not add personal data to free-text fields such as feedback or extension reasons.
Accuracy and correction	The administrator can edit names and departments; marks can only be corrected with a reason that is kept in the audit log.	Correct records when a learner or parent reports an error.
Access by the data subject	Students can see their own courses, every version they submitted and their released marks and feedback, but no other student's data (tested in StudentSeesOnlyOwnSubmission).	Answer access requests for data the console does not show, such as the audit log.
Retention	Deactivated users are kept so that past marks and the audit trail stay complete.	Set a retention period (for example, delete records some years after a learner leaves). v1.1 has no automatic deletion.
Security safeguards	Section 3.	Keep the computer and the backup drive locked away.
Notification	Not a system function.	Notify the Data Protection Commission once it is set up; it had not been appointed when this report was written.

Keeping the data on one school computer with no internet connection also helps: learners' data is never sent to a cloud service in another country, so the question of cross-border transfer does not arise.

- **Security of Assessment Records**

Marks decide learners' results, so they must stay confidential until release, must not be changed without a trace, and must not be lost.

Goal	Design measure	Evidence
Only the right people get in	Passwords stored as salted Argon2id hashes (libsodium); one message for a wrong user name or password; deactivated users cannot log in.	PasswordIsNotStoredInPlainText; DeactivatedUserCannotLogIn
People only do what their role allows	Every service checks the role and ownership: a lecturer only marks their own courses, an HoD only moderates their department.	OnlyTheCoursesLecturerMayMark; OnlyTheRightHodMayModerate
Marks stay confidential until checked	Students see a mark and feedback only after the HoD releases it (BR4).	StudentSeesNothingBeforeRelease
No silent changes	Every mark change is saved with an audit entry (who, when, old, new, reason) in one transaction; database triggers stop the audit log being edited or deleted.	AuditObserverWritesExactlyOneEntryPerChange; AuditLogCannotBeEdited
No tampering through input	All SQL uses bound parameters.	InjectionAttemptIsHarmless
Records are not lost	All data is in one file that is copied to a USB drive every day.	Installation guide, backups

Limits: the audit-log protection works through the program; someone who logs in to the computer itself could copy or replace the database file. The school should therefore give the LMS its own Windows account, limit who knows that password, and lock the backup drive away. The program also has no automatic log-out, so users must log out (option 0) before leaving the shared computer.

- **Accessibility for Users with Disabilities**

The Persons with Disability Equity Act, 2021 (Act 2 of 2021) requires educational institutions to take the steps needed to meet the special needs of learners with disabilities, including in examinations and class schedules (section 23), and gives persons with disabilities the right to information in an accessible format (sections 26 and 34).

- **What helps:** the program is plain text in a console window, so it can be read aloud by a screen reader such as Windows Narrator, and the console's zoom gives larger text. Everything is done with the keyboard; no mouse is needed. No information is shown only by colour. Every option is a number, and the common tasks need one menu choice after login (NFR9).
- **Reasonable accommodation:** a lecturer can give an individual extension, with a reason, to a learner who needs more time because of a disability. This is a direct way for the school to meet section 23.
- **What is still missing:** answers are one line of typed text, which is hard for learners who cannot type; dates must be typed in a fixed format; and the user manuals are not yet in Braille or audio. For such learners the school must keep another way to hand in work, and a later version should allow attaching files.
- **Fairness of the Late-Penalty Rule**

The penalty (10% of the maximum mark per day for up to three days, then zero) came from the Head of Department; the student representative asked that late work should not simply be rejected. Phase 2 resolved this conflict (C2), and the system applies the agreed rule.

- **Consistency:** the rule is applied by the program, the same way for every learner, with no room for a lecturer to be harsher with one learner than another. The raw mark, the days late, the penalty and the final mark are all stored and shown.
- **Transparency:** the rule is in the student manual, the console says how many days late a submission is, warns when work will receive zero, and the result shows the raw and final mark separately.
- **Late work is never rejected:** even work more than three days late is accepted and recorded, so the HoD can see it and the school can decide on special cases.
- **Room for good reasons:** individual and class-wide extensions (CR1) move the deadline before any penalty applies, and work already handed in late is recalculated when an extension is given.
- **Human check:** every mark is moderated by the HoD before release, who can adjust it with a recorded reason.

Two points are still open to debate. First, one second late counts as a full day, which is clear but strict; a short grace period could be considered. Second, the zero after three days is a large step compared with 30% on day three. The design keeps both in one replaceable class (StandardPenaltyStrategy), so the school can change the rule without changing the rest of the system. Finally, lateness depends on the computer's clock, so the technician should check it regularly; a wrong clock would be unfair to every learner.

- **Professional Responsibility**

The ACM/IEEE-CS Software Engineering Code of Ethics asks software engineers to act in the public interest, in the interest of their client, and to make their products meet the highest standards they can. In this project that meant:

Principle of the Code	**How it was applied**
Public: act in the public interest	Learners' data and fair marks were treated as requirements (NFR4–NFR6, NFR11), not optional extras.
Client and employer	Stakeholder conflicts were resolved openly in Phase 2, and the system does what the school agreed, not what was easiest to build.
Product: highest standards	117 automated tests, CI on every change, static analysis and a code review before each release.
Judgement: honesty	Limitations, open defects and differences from the design are reported in the documents instead of being hidden.

12. Conclusion

- **What was achieved**

Objective	**Result**
O1 Four roles	Met: one login screen and a separate menu per role; 100% of role-violation tests are denied.
O2 Business rules	Met: all 19 Must requirements and the Should requirements FR20–FR22 implemented; FR23 and FR24 (Could) dropped as planned in risk R12.

Objective	Result
O3 Audit and security	Met: Argon2id password hashes, parameterised SQL, and an append-only audit log written in the same transaction as each mark change.
O4 Waterfall process	Met: a deliverable for every phase, with the design checked before coding.
O5 Testing	Met: 117 automated tests at unit, integration and system level; 96% line coverage of the business logic against a target of 70%.
O6 CM and releases	Met: eight pull requests, each checked by CI; tags v1.0 and v1.1.

- **Evaluation**

The strongest part of the project is the link between the documents and the code. Every requirement can be traced to the classes that implement it and the tests that check it (Appendix A), and the design choices made early paid off later: keeping the effective-deadline rule in one function meant that change request CR1 needed only a small, low-risk change. The main weaknesses are the console interface, which allows only one-line answers and suits users who can type, and the single-computer design, which lets only one person use the system at a time. The database file is also only as safe as the computer it is on.

- **Lessons learned**
- **Review against the design, not only the tests.** All tests passed before a review against the SRS and the class diagram found that the Observer pattern and several use-case flows were missing. Passing tests show that the code does what the tests ask, not that it does what the design says.
- **Check that every merge really happened.** A network failure meant one slice was never pushed. Checking git log after every merge caught it, and it became a habit.
- **Small slices make progress visible.** Building one thin slice at a time, each with its own tests and pull request, kept the system working at every step and made problems easy to find.
- **Start early.** Starting the project late left little time for the final phases, so the deployment and maintenance work had to be done quickly at the end.
- **Future work**
- Filtering lists and exporting released marks to CSV (FR23, FR24).
- A network or web version so that several users can work at once.
- Attaching files to submissions, and automatic log-out on the shared computer.
- A retention rule that archives learners’ records after they leave, and a short grace period for late submissions if the school agrees.

13. References

- ACM/IEEE-CS Joint Task Force (1999) Software Engineering Code of Ethics and Professional Practice, version 5.2. New York: Association for Computing Machinery.
- Data Protection Africa (n.d.) Lesotho. Available at: https://dataprotection.africa/lesotho/
- Google (2023) GoogleTest User’s Guide, version 1.14. Available at: https://google.github.io/googletest/
- ISO/IEC/IEEE (2018) ISO/IEC/IEEE 29148:2018 Systems and software engineering — Life cycle processes — Requirements engineering. Geneva: ISO.

- Kingdom of Lesotho (2012) Data Protection Act, 2011 (Act No. 5 of 2012). Available at: https://lesotholii.org/akn/ls/act/2012/5
- Kingdom of Lesotho (2021) Persons with Disability Equity Act, 2021 (Act No. 2 of 2021). Available at: https://lesotholii.org/akn/ls/act/2021/2
- libsodium (n.d.) Password hashing: the pwhash API (Argon2). Available at: https://doc.libsodium.org/
- SQLite (n.d.) SQLite documentation. Available at: https://www.sqlite.org/docs.html

14. Appendices

Appendix A: requirements traceability matrix. Appendix B: fully dressed use cases. Appendix C: data dictionary. Appendix D: updated risk register. Appendix E: system test cases. Appendix F: installation guide. Appendix G: user manuals.

Appendix A: Requirements Traceability Matrix

Each requirement is traced to its use case, the design elements that realise it, the source files and the automated tests that verify it (release v1.1). Test names are as in the repository.

Req.	Use case	Design elements	Source files	Tests
FR1	UC01	AuthService, IPasswordHasher, User	AuthService.cpp, LibsodiumHasher.cpp	LoginSucceedsAndFailsCorrectly; WrongPasswordGivesOneGenericMessage
FR2	UC01	User::allowedActions, role subclasses, ConsoleApp menus	User.hpp, ConsoleApp.cpp	PermissionsDiffer; RolesOnlySeeTheirOwnMenu
FR3	UC02	AdminService, UserFactory, IUserRepository	AdminService.cpp, UserFactory.cpp, SqliteUserRepository.cpp	CreatesUsersOfAllRolesWhoCanThenLogIn; EditsUserProfile; DeactivatedUserCannotLogIn
FR4	UC03	AdminService, Department, Course	AdminService.cpp, SqliteCourseRepository.cpp	CreatesCoursesAndRejectsDuplicates; AdminRenamesDepartment
FR5	UC04	AdminService, Course, enrolment	AdminService.cpp, SqliteCourseRepository.cpp	AssignsOnlyLecturersToCourses; EnrolsOnlyActiveStudentsOnce
FR6	UC05	AssignmentService, Assignment	AssignmentService.cpp, SqliteAssignmentRepository.cpp	LecturerCreatesAssignmentForOwnCourse; RejectsInvalidAssignmentInput
FR7	UC06	AssignmentService, Extension	AssignmentService.cpp, SqliteAssignmentRepository.cpp	ExtensionChangesOnlyThatStudentsDeadline; ExtensionRulesAreEnforced
FR8	UC12	AssignmentService, AssignmentView	AssignmentService.cpp	StudentSeesOwnCoursesAndEffectiveDeadlines; StudentListShowsKindOfExtensionAndOriginalDeadline
FR9	UC13	SubmissionService, Submission, ISubmissionRepository	SubmissionService.cpp, SqliteSubmissionRepository.cpp	OnTimeSubmissionIsRecordedWithTimestamp; LateDaysBoundaryValues; VeryLateWorkIsStillAccepted
FR10	UC14	SubmissionService, SubmissionVersion	SubmissionService.cpp	ResubmissionKeepsAllVersions; NoResubmissionAfterTheDeadline

Req.	Use case	Design elements	Source files	Tests
FR11	UC15	IPenaltyStrategy, StandardPenaltyStrategy, Submission::finalMark	IPenaltyStrategy.cpp, Submission.cpp	SrsExampleTable; LatePenaltyTable
FR12	UC15	IPenaltyStrategy::isZeroMark	IPenaltyStrategy.cpp	NeverBelowZeroAndRounds; VeryLateWorkIsStillAccepted
FR13	UC07, UC08	MarkingService	MarkingService.cpp	LecturerMarksSubmissionWithFeedback; MarkBoundaryValues; FeedbackIsRequiredAndLimited
FR14	UC07, UC09, UC11	Submission::transitionTo, SubmissionState, StatusObserver	Submission.cpp	OnlyAllowedTransitions; FullLifeCycle
FR15	UC09	ModerationService::listForModeration	ModerationService.cpp	HodSeesMarkedWorkFromOwnDepartmentOnly; ModerationListShowsLecturerAndExtension
FR16	UC09, UC10	ModerationService::approve, adjustMark, approveAll	ModerationService.cpp	ApproveKeepsMarkAndIsAudited; AdjustChangesMarkWithReason; ApproveAllApprovesOnlyMarkedWork
FR17	UC11	ModerationService::release	ModerationService.cpp	ReleaseOnlyReleasesModeratedWork; ReleaseNamesThePendingSubmissions
FR18	UC16	ModerationService::resultFor	ModerationService.cpp, ConsoleApp.cpp	StudentSeesNothingBeforeRelease; StudentSeesFinalMarkAndFeedbackAfterRelease
FR19	UC17	ISubmissionObserver, AuditObserver, IAuditRepository, AuditEntry	SqliteAuditRepository.cpp, MarkingService.cpp, ModerationService.cpp	AuditTrailRecordsWhoWhenOldNewAndWhy; AuditObserverWritesExactlyOneEntryPerChange
FR20	UC17	MarkingService::auditTrail, IAuditRepository	MarkingService.cpp	AuditTrailVisibility
FR21	UC07	SubmissionService::statusForAssignment, StatusObserver	SubmissionService.cpp	LecturerSeesStatusOfEveryEnrolledStudent; StatusObserverSavesTheNewState
FR22	UC02	AdminService::resetPassword	AdminService.cpp	ResetPasswordChangesWhichPasswordWorks
FR23	UC12	Not implemented (Could)	-	-
FR24	UC11	Not implemented (Could)	-	-

Req.	Use case	Design elements	Source files	Tests
FR25	UC06	AssignmentService::grantClassExtension, ClassExtension	AssignmentService.cpp, SqliteAssignmentRepository.cpp	ClassExtensionMovesEveryStudentsDeadline; ClassExtensionRules
NFR1	-	AuthService, database indexes	schema.sql	LoginWithin2Seconds
NFR2	-	Database indexes	schema.sql	StudentAssignmentListWithin1Second; ModerationListOf800Within3Seconds
NFR3	-	SQLite file, prepareDatabase	DatabaseSetup.cpp	CapacityIsStoredWithoutErrors; DataSurvivesRestartAndAdminIsNotRecreated
NFR4	-	IPasswordHasher, LibsodiumHasher	LibsodiumHasher.cpp	PasswordIsNotStoredInPlainText; SaltedHashAndVerify
NFR5	-	Sqlite*Repository classes, Statement	Sqlite*.cpp, DatabaseConnection.cpp	InjectionAttemptIsHarmless; code review (Phase 6)
NFR6	-	User::can and ownership checks in every service	All service files	OnlyTheRightHodMayModerate; OnlyEnrolledStudentsMaySubmit
NFR7	-	Errors.hpp, InputValidator, ConsoleApp	InputValidator.cpp, ConsoleApp.cpp	BadInputAndRuleErrorsDoNotStopTheProgram; ParsesNumbersAndDates
NFR8	-	Single-computer architecture, SQLite	-	Whole suite run with networking disabled
NFR9	-	ConsoleApp role menus	ConsoleApp.cpp	RolesOnlySeeTheirOwnMenu; inspection
NFR10	-	CI quality job, coding standard	ci.yml, CMakeLists.txt	Coverage gate (96%)
NFR11	-	AuditObserver, ITransactionRunner, audit triggers	schema.sql, DatabaseConnection.cpp	AuditLogCannotBeEdited; FailingObserverRollsBackTheWholeChange

Appendix B: Fully Dressed Use Cases

UC13 Submit assignment (extended by UC14 Resubmit and UC15 Apply late penalty)

Field	Description
Primary actor	Student
Preconditions	The student is logged in, enrolled in the course, and the assignment exists.
Main flow	1. The system lists the student's assignments with effective deadlines. 2. The student selects an assignment. 3. The system checks for an earlier submission. 4. The student enters the work. 5. The system compares the time with the effective deadline (BR2); it is on time. 6. The system saves the submission with a timestamp, state Submitted. 7. The system confirms and shows the time received.
Alternative flows	A1 Resubmission before the deadline: a new version is stored, earlier ones kept (BR3). A2 Late within 3 days: state Late; the student is told how many days late. A3 Beyond 3 days: still accepted as Late; the student is warned the mark will be zero. A4 Extension: the extended deadline is used (BR2).
Exception flows	E1 Not enrolled or no such assignment: error, back to the menu. E2 Empty input: message, nothing saved. E3 Late resubmission: refused, late work is final. E4 Database error: nothing saved, no crash. E5 Already marked: resubmission refused.
Postconditions	The latest version is stored with its time and state; earlier versions kept.
Requirements	FR8–FR12, BR1–BR3, NFR6, NFR7

UC07 Mark submission (includes UC08 Write feedback and UC17 Record audit entry)

Field	Description
Primary actor	Lecturer
Preconditions	The lecturer is logged in and teaches the course; a submission is Submitted or Late.
Main flow	1. The system lists the submissions with their states. 2. The lecturer selects one. 3. The system shows the work, the maximum mark and any penalty. 4. The lecturer enters a raw mark from 0 to the maximum. 5. The lecturer writes feedback. 6. The system applies the penalty (BR1). 7. The mark and feedback are saved; state Marked. 8. An audit entry is recorded (UC17). 9. The system confirms "Marked, awaiting moderation".
Alternative flows	A1 Late work: the deduction is shown. A2 Re-marking before moderation: a reason is required and audited. A3 Beyond 3 days: final mark zero; feedback still allowed.

Field	Description
Exception flows	E1 Mark out of range or not a number: rejected. E2 Already moderated: refused. E3 Not the course's lecturer: access denied. E4 Database error: nothing saved, no crash.
Postconditions	State Marked; mark, feedback and audit entry stored; students cannot see the mark.
Requirements	FR13, FR14, FR19, FR21, NFR6, NFR7, NFR11

UC09 Moderate marks (UC10 Adjust mark; UC11 Release approved marks)

Field	Description
Primary actor	Head of Department
Preconditions	The HoD is logged in; the department has at least one Marked submission.
Main flow	1. The system lists the department's Marked submissions with lecturer, mark, penalty and extension (FR15). 2. The HoD reviews one. 3. The HoD approves it. 4. State Moderated; approver and time recorded. 5. Steps 2–4 repeat until all are moderated (BR5). 6. The HoD releases the assignment. 7. The system checks all are Moderated, sets them Released and records the time. 8. The system confirms how many were released (BR4).
Alternative flows	A1 Adjust mark (UC10): new mark with a mandatory reason; audit entry. A2 Bulk approval: all remaining submissions approved, one audit entry each. A3 Partial release: only the moderated ones are released.
Exception flows	E1 Course not in the HoD's department: access denied. E2 Adjustment without a reason: refused. E3 Release while some are not moderated: pending ones listed, nothing released. E4 Mark out of range: rejected. E5 Database error: no partial change, no crash.
Postconditions	Approved work is Moderated or Released; every adjustment audited.
Requirements	FR14–FR20, NFR2, NFR6, NFR11

Appendix C: Data Dictionary (release v1.1)

Table	Column	Type and constraints	Description
department	department_id	INTEGER PK	Key
	name	TEXT NOT NULL UNIQUE	Department name
app_user	user_id	INTEGER PK	Key
	username	TEXT NOT NULL UNIQUE	Login name
	password_hash	TEXT NOT NULL	Argon2id hash with embedded salt
	full_name	TEXT NOT NULL	Display name
	role	TEXT CHECK (ADMIN, LECTURER, HOD, STUDENT)	Which User subclass
	is_active	INTEGER 0/1, default 1	Deactivated users cannot log in
	department_id	INTEGER FK department	For lecturers and HoDs; one HoD per department
course	course_id	INTEGER PK	Key
	code	TEXT NOT NULL UNIQUE	Course code
	name	TEXT NOT NULL	Title
	department_id	INTEGER FK NOT NULL	Owning department
	lecturer_id	INTEGER FK	Assigned lecturer
enrolment	student_id	INTEGER FK, composite PK	Student
	course_id	INTEGER FK, composite PK	Course
assignment	assignment_id	INTEGER PK	Key
	course_id	INTEGER FK	Course
	title	TEXT NOT NULL	Title
	deadline	TEXT (UTC ISO-8601)	Original deadline
	max_mark	INTEGER > 0	Maximum mark
extension	assignment_id, student_id	FKs, UNIQUE together	One per student and assignment
	new_deadline	TEXT	The student’s deadline
	granted_by, granted_at	FK, TEXT	Who and when
	reason	TEXT NOT NULL	Why

Table	Column	Type and constraints	Description
class_extension (v1.1)	assignment_id	INTEGER PK, FK	One per assignment (CR1)
	new_deadline	TEXT	Deadline for the whole class
	granted_by, granted_at	FK, TEXT	Who and when
	reason	TEXT NOT NULL	Why
submission	submission_id	INTEGER PK	Key
	assignment_id, student_id	FKs, UNIQUE together	One per student and assignment
	state	TEXT CHECK (5 states)	Life-cycle state
	submitted_at	TEXT	Time of latest version
	days_late	INTEGER >= 0	Started 24-hour periods late
	raw_mark	INTEGER >= 0 or NULL	Mark before penalty
	feedback	TEXT	Hidden until released
	marked_by, moderated_by	FKs	Lecturer and HoD
	released_at	TEXT	Release time
submission_version	version_id	INTEGER PK	Key
	submission_id	INTEGER FK	Owning submission
	version_no	INTEGER >= 1, UNIQUE with submission	1, 2, 3 …
	content	TEXT NOT NULL	The work
	submitted_at	TEXT	When handed in
audit_log	audit_id	INTEGER PK	Key
	submission_id	INTEGER FK	Submission
	changed_by, changed_at	FK, TEXT	Who and when
	old_value, new_value	INTEGER	Raw mark before and after
	reason	TEXT, not blank	Why; triggers block UPDATE and DELETE

Appendix D: Updated Risk Register

ID	Risk	L×I	What happened	Status
R1	Late start and compressed schedule	20	Occurred. Must requirements built first; FR23 and FR24 dropped as planned	Closed
R2	Scope creep from conflicting wishes	12	Controlled: conflicts resolved in the SRS; only CR1 added, through the change process	Closed
R3	Loss of work	15	A network failure left one slice unpushed; found with git log, no work lost	Closed
R4	Unfamiliar tools (SQLite C API, CMake, CI)	12	GoogleTest failed to build on the Windows drive; fixed by building in the Linux home folder	Closed
R5	Insecure passwords or SQL injection	10	Did not occur: Argon2id and parameterised queries, both tested	Closed
R6	Poor performance on old hardware	6	Did not occur: login 0.25 s, lists 0.2 s with full data	Closed
R7	Ambiguous stakeholder input	9	Assumptions recorded as resolutions C1–C12	Closed
R8	Defects found late	12	Partly occurred: the review before release found the Observer missing; fixed and retested	Closed
R9	Code and design drift apart	9	Occurred; differences found and documented (Section 6)	Closed
R10	CR1 larger than expected	6	Did not occur: about 220 lines in 17 files	Closed
R11	Illness or personal disruption	10	Did not occur	Closed
R12	Design larger than the time for coding (added at Gate 2)	12	Partly occurred: Could items dropped	Closed

Appendix E: System Test Cases

One system test case was written for every flow of the use cases specified in Phase 2. Each case is carried out by the automated test named in the last column, so it is re-run on every push.

ID	Flow	Expected result	Automated test	Result
ST-01	UC01 wrong username or password	Same message for both; not logged in	WrongPasswordGivesOneGenericMessage	Pass
ST-02	UC13 main: submit on time	Saved with time, state Submitted	OnTimeSubmissionIsRecordedWithTimestamp	Pass
ST-03	UC13 A1 resubmit before deadline	New version saved, old ones kept	ResubmissionKeepsAllVersions	Pass
ST-04	UC13 A2 late within 3 days	Accepted as Late; days late shown	LateSubmissionIsAcceptedAndPenalised	Pass
ST-05	UC13 A3 more than 3 days late	Still accepted, marked Late	VeryLateWorkIsStillAccepted	Pass
ST-06	UC13 A4 extension	Lateness counted from the extension	ExtensionIsUsedForLateness	Pass
ST-07	UC13 E1 not enrolled or no such assignment	Access denied / not found; nothing saved	OnlyEnrolledStudentsMaySubmit	Pass
ST-08	UC13 E2 empty input	Rejected with a message	RejectsEmptyOrTooLongContent	Pass
ST-09	UC13 E3 late resubmission	Refused; first version kept	NoResubmissionAfterTheDeadline	Pass
ST-10	UC13 E4 database error	Nothing saved; error reported	VersionAndSubmissionAreSavedTogether	Pass
ST-11	UC13 E5 work already marked	Resubmission refused	NoResubmissionAfterMarking	Pass
ST-12	UC07 main: mark with feedback	State Marked; audit entry “initial marking”	LecturerMarksSubmissionWithFeedback	Pass
ST-13	UC07 A1 late submission	Penalty deducted from raw mark	LatePenaltyTable	Pass
ST-14	UC07 A2 re-mark before moderation	Reason required; old and new value audited	ChangeMarkNeedsReasonAndUpdatesMark	Pass
ST-15	UC07 A3 more than 3 days late	Final mark 0; feedback still saved	LatePenaltyTable	Pass
ST-16	UC07 E1 mark out of range or not a number	Rejected with a message	MarkBoundaryValues; BadInputAndRuleErrorsDoNotStopTheProgram	Pass
ST-17	UC07 E2 already moderated	Change refused	LecturerCannotChangeMarkAfterModeration	Pass
ST-18	UC07 E3 lecturer not on the course	Access denied	OnlyTheCoursesLecturerMayMark	Pass

ID	Flow	Expected result	Automated test	Result
ST-19	UC07 E4 database error	Neither mark nor audit entry saved	MarkIsNotSavedIfAuditFails	Pass
ST-20	UC09 main: approve, UC11 release	Moderated then Released; student sees the mark	FullLifeCycle	Pass
ST-21	UC09 A1 / UC10 adjust mark	New mark saved with reason in the audit log	AdjustChangesMarkWithReason	Pass
ST-22	UC09 A2 bulk approval	One approval and one audit entry each	ApproveAllApprovesOnlyMarkedWork	Pass
ST-23	UC09 A3 partial release	Only moderated work released	ReleaseOnlyReleasesModeratedWork	Pass
ST-24	UC09 E1 other department	Access denied	OnlyTheRightHodMayModerate	Pass
ST-25	UC09 E2 adjust without a reason	Refused	AdjustChangesMarkWithReason	Pass
ST-26	UC09 E3 release with work pending	Refused; pending submissions listed	ReleaseNamesThePendingSubmissions	Pass
ST-27	UC09 E4 mark out of range; E5 database error	Rejected; nothing changed	AdjustBoundaryValues; ModerationIsNotSavedIfAuditFails	Pass

Appendix F: Installation Guide

This guide is for the school technician. It takes about 30 minutes. An internet connection is needed only during installation; after that the system works offline.

- **What you need**
- **Computer:** Windows 10 (version 2004 or later) or Windows 11, 64-bit, at least 4 GB of RAM and 2 GB of free disk space. A computer that already runs Ubuntu 22.04 or later can skip Step 1.
- **Account:** a Windows administrator account (for Step 1 only).
- **Backup:** a USB drive for daily backups.
- **Installing**

Step 1. Install Ubuntu inside Windows. Open PowerShell as administrator and run the command below, then restart the computer. When Ubuntu opens, choose a Linux user name and password.

```
wsl --install -d Ubuntu
```

Step 2. Install the build tools and libraries. In the Ubuntu window run:

```
sudo apt update
sudo apt install -y build-essential cmake pkg-config git libsqlite3-dev libsodium-dev
```

Step 3. Download release v1.1 into your home folder. Keep this folder: the program reads schema.sql from it, so it must not be moved or deleted.

```
git clone --branch v1.1 https://github.com/lelia-605/cs4433-lms.git ~/cs4433-lms
```

Step 4. Build the program and run the tests. Build in the home folder (~), not under /mnt/c, because building on the Windows drive fails with “Permission denied”.

```
cd ~/cs4433-lms
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build --parallel
ctest --test-dir build
```

The last line must say “100% tests passed, 0 tests failed out of 117”. If it does not, do not use the system; see the troubleshooting table below.

Step 5. Make a folder for the data and start the program for the first time.

```
mkdir -p ~/lms-data
cd ~/lms-data && ~/cs4433-lms/build/lms
```

The program prints “New database created” and creates the first administrator: user name admin, password admin12345. Log in, choose 5 “Reset a password” and give the admin account (user id 1) a new password at once.

Step 6. Optional: make a desktop shortcut. In Windows, right-click the desktop, choose New > Shortcut and enter the line below. If the Ubuntu name is different, check it with wsl -l in PowerShell.

```
wsl.exe -d Ubuntu -- bash -lc "cd ~/lms-data && ~/cs4433-lms/build/lms"
```

- **Backups and restoring**

All data is in one file, ~/lms-data/lms.db. Copy it to the USB drive at the end of each day, when nobody is logged in. If the USB drive is drive E: in Windows, run:

```
cp ~/lms-data/lms.db /mnt/e/lms-backup-$(date +%F).db
```

To restore, close the program and copy the backup back over ~/lms-data/lms.db. Backups contain learners' personal data and marks, so the USB drive must be kept locked away.

- **Upgrading from v1.0 to v1.1**

Back up lms.db first. Then download and build the new release; the first time v1.1 starts it prints "Database upgraded to version 1.1" and keeps all existing data.

```
cd ~/cs4433-lms && git fetch --tags && git checkout v1.1
cmake --build build --parallel && ctest --test-dir build
```

- **Troubleshooting**

Problem	Cause	What to do
"Package 'libsodium' not found" when running cmake	Library not installed	Run Step 2 again
"Could not resolve host: github.com"	No internet connection	Reconnect and repeat the step; nothing was changed
"Permission denied" while building	Building on the Windows drive (/mnt/c)	Use the home folder, as in Step 3
"Cannot start: cannot read script"	The ~/cs4433-lms folder was moved or deleted	Put it back or repeat Steps 3 and 4
"Cannot start: cannot open database"	The data folder does not exist or is read-only	Run mkdir -p ~/lms-data
Nobody can log in as administrator	Admin password forgotten	Another administrator resets it; otherwise restore the last backup

Appendix G: User Manuals

- **For every user**
- **Starting:** open the LMS shortcut (or run the command from Step 5). Type your user name and password. A wrong user name and a wrong password give the same message, “Wrong username or password”.
- **Menus:** type the number of an option and press Enter. Ids (user id, course id, assignment id) are shown by the “List” options. Type 0 to log out, then quit as the user name to close the program.
- **Dates:** type them as YYYY-MM-DD HH:MM, for example 2026-11-20 10:00. Times are in UTC, which is 2 hours behind Lesotho time, so 10:00 UTC is 12:00 in Maseru.
- **Messages:** “Not done” means a rule stopped the action, “Access denied” means your role may not do it, and “Not found” means a wrong id. Nothing is saved, and the menu carries on.
- **Administrator**

Set up a new term in this order, because each step needs ids from the one before.

Option	Use it to	You are asked for
7 Create department	Add a department, e.g. Science	Name
2 Create user	Add a lecturer, HoD or student	Username, password (8+ characters), full name, role 1–4, department id for lecturers and HoDs
9 Create course	Add a course	Course code, name, department id
11 Assign lecturer to course	Say who teaches it	Course id, lecturer user id
12 Enrol student in course	Put a student in a course	Student user id, course id
1, 6, 8 List …	See users, departments, courses and their ids	-
3 Edit user / 13 Rename department / 10 Rename course	Correct a name or department	Id and the new value
4 Deactivate user	Stop someone logging in (records are kept)	User id
5 Reset a password	When a user forgets their password	User id, new password
14 Audit trail for a submission	See every change to a mark	Assignment id, student user id

- **Lecturer**

Option	Use it to	You are asked for
1 My courses / 2 Assignments in a course	See your courses and assignments with their deadlines	Course id
3 Create assignment	Set a new assignment	Course id, title, deadline, maximum mark (1–1000)

Option	Use it to	You are asked for
4 Grant an extension	Give one student more time	Assignment id, student user id, new deadline, reason
10 Extend the deadline for the whole class	Give every student more time	Assignment id, new deadline, reason
5 Submission status	See who has submitted, late or on time, and versions	Assignment id
6 Mark a submission	Read the work, then give a mark and feedback; the late penalty is applied for you	Assignment id, student user id, mark, feedback
7 Change a mark	Correct a mark before the HoD moderates it	As above plus a reason
8 View a mark / 9 Audit trail	Check a mark and its history	Assignment id, student user id

Students cannot see your mark until the Head of Department has moderated and released it. After moderation only the HoD can change a mark.

- **Head of Department**

Option	Use it to	You are asked for
1 Marks waiting for moderation	See marked work in your department with raw mark, penalty, final mark, lecturer and any extension	-
2 Approve a mark	Accept one mark as it is	Submission id
3 Adjust a mark	Change a mark; the reason is saved in the audit log	Submission id, new mark, reason
4 Approve all marks for an assignment	Accept every marked submission at once	Assignment id
5 Assignments in my department	Find assignment ids	-
6 Release marks for an assignment	Let students see their marks and feedback	Assignment id
7 Audit trail for a submission	See who changed a mark, when and why	Assignment id, student user id

If some work for the assignment is not moderated yet, release is refused and the pending submissions are listed. You can then answer y to release only the moderated marks.

- **Student**

Option	Use it to	You are asked for
1 My courses	See the courses you are enrolled in	-

Option	Use it to	You are asked for
2 Assignments in a course	See deadlines; an extension shows whether it is for the whole class or only you, with the original date	Course id
3 Submit or resubmit	Hand in your answer (one line of text)	Assignment id, your answer
4 My submitted versions	See every version you handed in	Assignment id
5 My result	See your mark and feedback once released	Assignment id

- **Resubmitting:** you may resubmit as often as you like before your deadline; every version is kept and the latest is marked.
- **Late work:** is always accepted. You lose 10% of the maximum mark for each day late, for up to 3 days; after 3 days the mark is zero. A submission one second late counts as one day late.
- **Results:** say “not released yet” until the Head of Department has checked them.